

Five research items from the technical review

Findings for the items assigned at the 28 July review — the full-stack comparison, database independence, system architecture and the mobile framework — plus the cloud-provider comparison that was missing from the last report. Each is compared against the six criteria agreed for this project, with a summary matrix. **The mobile finding reverses the recommendation given verbally on 28 July; the reason is set out in item four.**

THE SIX CRITERIA 1 Token efficiency to build · 2 Token efficiency to maintain · 3 Scalability · 4 Structural stability and complexity · 5 Cloud-provider independence · 6 AI tooling and knowledge base

ITEM ONE

Full-stack comparison, compared as whole stacks

The original evaluation compared each layer separately. As requested, the comparison was re-run as two complete stacks, because the choice of front end could reasonably have forced the choice of back end.

CRITERION	OPTION A — GO + NEXT.JS + MOBILE	OPTION B — NODE.JS/TYPESCRIPT + NEXT.JS + MOBILE
1 · Build effort	Moderate to high. One clear way to do most things, but more boilerplate to write.	Better. Most concise, and the largest body of examples for AI to draw on.
2 · Maintenance effort	Slightly better. Small, stable language; little churn.	Good. Type checking helps, but the package ecosystem moves faster.
3 · Scalability	Better. Higher raw capacity for heavy concurrent work.	Sufficient for our targets — 50,000 signed-in sessions per region.
4 · Stability / complexity	Better. Very small language surface, stable tooling.	Good, but package churn is a real ongoing cost.
5 · Cloud independence	Full — compiles to a small self-contained program.	Full — runs in a standard container.
6 · AI knowledge base	Good.	Best of any option. The single largest advantage either stack has.

OUTCOME — NO CHANGE

Comparing whole stacks does not change the answer. The front end is the same in both options, so the decision still rests on the back end alone, and **Option B wins on the AI knowledge base** — the criterion that matters most when AI writes and maintains the code. Go's genuine advantages in raw capacity and language simplicity do not close that gap. Go remains the sensible fallback if one specific part of the system is later measured to need it.

ITEM TWO

Keeping the platform independent of any one database

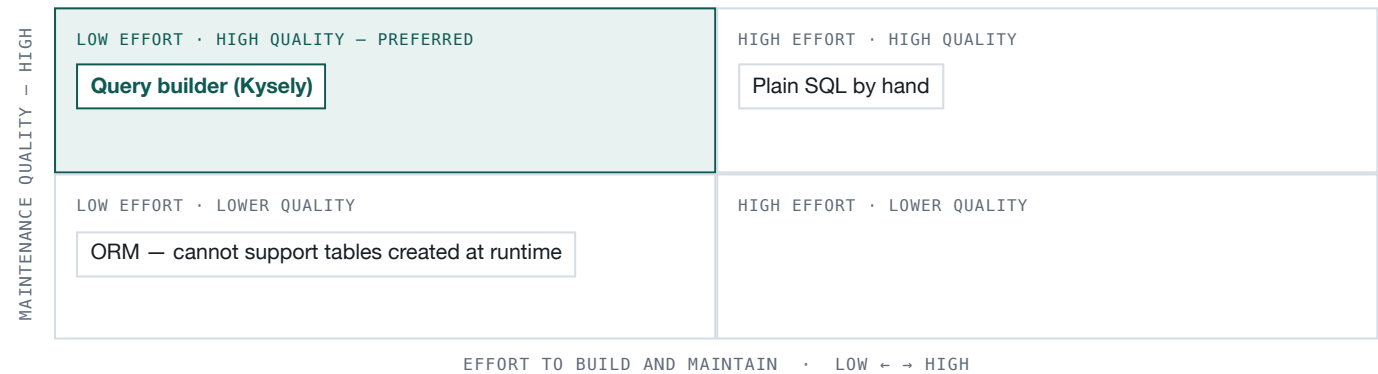
Objective: AI ahaMatic must work with, and be able to move between, any SQL database, because the choice is not final and may change. Scope confirmed as SQL only.

The finding that shapes this

The usual tool for this job is an ORM or ODM — a layer that maps program objects onto database tables. **It does not fit us, for a specific reason:** those tools need to know your data structures before you ship, but on our platform *builders create their own tables while the platform is running*. There is nothing to map in advance.

What we actually need is a **query builder** — a layer that helps write correct SQL and handles the differences between database engines, without assuming the tables are known upfront. It delivers the independence you asked for; it just isn't the category of tool named.

CRITERION	QUERY BUILDER (KYSELY)	ORM (PRISMA, TYPEORM, SEQUELIZE)	PLAIN SQL BY HAND
1 · Build effort	Low. Concise, close to SQL.	Low — best convenience of the three, where it applies.	High. Everything written out longhand.
2 · Maintenance effort	Low. The SQL stays readable, so changes are easy to review.	Moderate — heavy layer to reason through.	High — no automatic checking; every error becomes a review problem.
3 · Scalability	No limitation — full control over the queries.	Can hide inefficient queries.	No limitation.
4 · Stability / complexity	Thin layer, little to go wrong.	Largest layer, most historical churn.	Thin, but all the risk sits in our own code.
5 · Database & cloud independence	Supports Postgres, MySQL and SQL Server.	Supported, but tied to its own schema definition.	Full, but every difference is ours to handle.
6 · AI knowledge base	Good and growing.	Largest — but unusable here (see above).	Largest of all — SQL itself.



Which database as the starting point

Postgres — strictest data checking, best support for flexible structures (which matters when builders invent their own), free, and available as a managed service on every cloud. **SQL Server** is genuinely capable but carries licence cost and pulls toward one vendor's ecosystem. **MySQL/MariaDB** stays supported but is weaker on data checking. **Oracle** was excluded on cost.

Record identifiers: UUIDv7 rather than auto-numbering — they can be created offline or on a device, they keep the database fast because they sort by time, and they work identically on all three engines. Auto-numbering uses different syntax per engine, cannot be generated offline, and reveals how many records exist.

THE COST OF STAYING INDEPENDENT

The more portable we stay, the less we can use. If the platform must behave identically on MySQL, we cannot rely on Postgres's stronger data checking or its row-level security. Postgres becomes the engine we develop against, not one whose special features we build on. **Also worth separating:** writing portable queries is one problem; moving live data between engines later is a different one, because data types differ. Portable code makes that migration easier but does not remove it.

ONE CONFLICT FOUND

The standard way to keep tenants separate is the database's row-level security feature — but **MySQL does not have it**, so relying on it would break the independence goal. Recommending instead that **each tenant gets its own schema**, which works on all three engines and keeps tenants separated by structure rather than by a filter our code has to remember to apply.

System architecture

All four options were considered openly, not just the monolith.

CRITERION	MODULAR MONOLITH	MONOLITH, EXTRACT LATER	MICROSERVICES	SERVERLESS
1 · Build effort	Lowest — one codebase, one deployment.	Same at the start.	Highest — each service needs its own setup and contracts.	High — heavy configuration per function.
2 · Maintenance effort	Lowest — a change is one repository, one test run.	Low, rising with each part split off.	Highest — one change can touch many services.	High — fragmented and hard to see as a whole.
3 · Scalability	Adequate — add more copies. Weakness: everything scales together.	Better — scale only what measurement shows needs it.	Best — each part scales on its own.	Good for sudden spikes; poor for steady database-heavy work.
4 · Stability / complexity	Simplest — matches our seven fixed components exactly.	Low, rising per split.	Most complex — partial failures and network problems to handle.	Complex — start-up delays, vendor-specific behaviour.
5 · Cloud independence	Best — one container, runs anywhere.	Strong.	Moderate — orchestration choices tie you down over time.	Weakest — most locked to one provider.
6 · AI knowledge base	Strong.	Strong.	Patterns well documented, but diagnosing faults across services is where AI is weakest.	Moderate.



RECOMMENDATION

One deployable unit, organised internally as modules matching our seven components, with automated checks that enforce the boundaries between them on every change. Microservices win on exactly one criterion — scalability — and that can be bought later by splitting off a specific part once measurement proves it is needed, without paying the complexity cost now. There is a second reason: our specification already fixes which components may depend on which, and inside one codebase a tool can *prove* those rules automatically on every change. Across separate services, nothing can. To be clear, this is not simply continuing the old platform's monolith — the enforced boundaries are the difference.

A SEPARATE QUESTION WORTH DECIDING ON ITS OWN

How the **platform** is structured is a different question from **how builders' applications actually run**. Those applications likely need their own isolated process or container per tenant — for security, resource limits, and to contain failures — whichever way the platform itself is organised. This report does not decide that.

ITEM FOUR

Mobile framework — corrected after checking sources

On 28 July I argued verbally for Flutter, on three claims about the state of each ecosystem. Those claims were then checked against current published sources. **Three of them do not hold, and the recommendation reverses.**

WHAT THE CHECKS CONTRADICTED

- 1 • **"Most device features are first-party in Flutter."** Not supported — and closer to the reverse for what we need. Flutter's official set covers camera and biometrics, but **not** secure storage, location, or barcode scanning; those are community packages, and two of the three are maintained by **the same individual**. Expo ships all three as first-party. The leading Flutter scanner package exists only because the previous one was abandoned when its underlying libraries stopped being maintained.
- 2 • **"React Native's internal rewrite is still causing churn."** Largely out of date. The new architecture has been the default since version 0.76 and the old one was removed entirely in 0.82 — we would be starting on 0.86. Around 90% of the main ecosystem is compatible, and it runs in production at Facebook, Instagram and Messenger.
- 3 • **"Three build systems must agree, which is near an AI's worst case."** This was a real problem, and it was fixed. Expo's SDK 54 overhauled how native dependencies are resolved, added duplicate detection, and now ships pre-compiled modules on its build service.

CRITERION	REACT NATIVE + EXPO	FLUTTER
1 • Build effort	Lower. Same language as our server and web; largest body of examples.	Moderate. A second language, fewer examples.
2 • Maintenance effort	Comparable or better. Expo maintains the device features we need as first-party, funded by a company with a commercial product behind it.	Good, but secure storage, location and scanning rest on volunteer maintainers — two of them the same person.
3 • Scalability	Not a factor — runs on the phone.	Not a factor.
4 • Stability / complexity	Good. Architecture migration is settled; dependency resolution substantially fixed.	Better on one point: draws its own screen, so iOS and Android match by construction and an OS update cannot shift our layout.
5 • Cloud independence	Not applicable.	Not applicable.
6 • AI knowledge base	Largest of any option.	Smaller, though well documented.



An over-claim I withdraw. I said a single rendering engine was needed to meet our requirement that a built application's mobile version match its web version. That was wrong. The requirement is parity of *guarantees* — separation between tenants, access control, data integrity — not identical pixels. Visual consistency across iOS and Android is a real benefit, but it is a quality gain, not something our specification demands.

RECOMMENDATION — REACT NATIVE WITH EXPO

One language across server, web and mobile, the largest knowledge base for AI to draw on, and better first-party coverage of the device features we actually need than Flutter offers. **Flutter keeps one genuine advantage** — consistent appearance across platforms — and would be the better choice for a design-led consumer product. It is worth less to us than a single language and the largest body of examples. Note also that this is React Native *with Expo*, not bare React Native; without Expo most of the maintenance argument reverses.

WHAT THIS EPISODE IS WORTH TAKING FROM IT

The original concern about React Native was legitimate — the ecosystem genuinely was unsettled, which is presumably where its reputation came from. It has since been addressed, and checking is what established that. The lesson recorded for future decisions: **claims about how well an ecosystem is maintained go out of date faster than anything else we compare, and must be checked against current sources before they are allowed to decide anything.**

ITEM FIVE

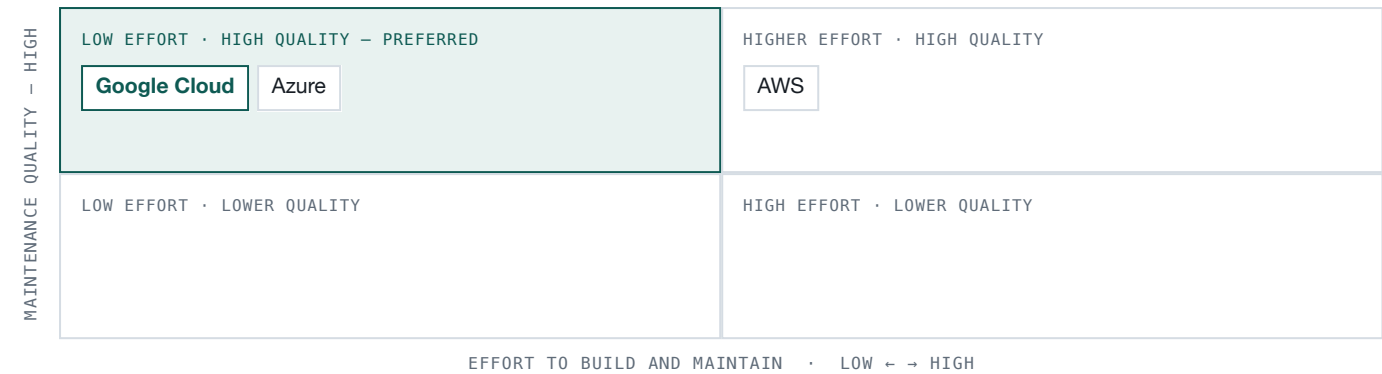
Cloud provider — the comparison missing from the last report

The previous report named Google Cloud as a reference target without comparing it to anything. This is that comparison.

THE FACT THAT BOUNDS THIS DECISION

Container images are portable across all three providers. The configuration around them is not. Moving provider is not a rewrite of the application — it is a rewrite of infrastructure definitions, networking, access policy and deployment pipelines. That limits how much any choice can trap us, and it says exactly where the switching cost sits. Two rules follow, and they matter more than which provider we pick: define infrastructure in a provider-neutral tool rather than a provider's own console, and use only the portable subset — containers, managed database, file storage — with nothing provider-specific that correctness depends on.

CRITERION	GOOGLE CLOUD	AWS	AZURE
1 · Build effort	Lowest. Simplest path to running a container that serves web requests.	Highest. More surrounding pieces to configure, though most documented.	Moderate — similar model to Google's.
2 · Maintenance effort	Lowest — fewest moving parts to keep working together.	Highest — most components and most access policy to maintain.	Moderate.
3 · Scalability	Sufficient — meets our 50,000-session target and scales down to nothing when idle.	Deepest at very large scale, and cheapest for always-on capacity.	Sufficient.
4 · Stability / complexity	Lowest complexity for the shape we are building.	Highest, in exchange for the most control.	Moderate.
5 · Independence	Equal in practice — the application is portable from all three; the infrastructure configuration is specific to each. Kubernetes on any provider is the most portable option but adds complexity we do not currently need.		
6 · AI knowledge base	Good.	Largest by a clear margin.	Good.
Managed database cost	Lowest of the three at a representative production size.	Middle.	Highest of the three at a comparable size.



RECOMMENDATION – GOOGLE CLOUD AS THE DEFAULT, NEITHER ALTERNATIVE EXCLUDED

It wins on the criteria that bind us: least effort to build and maintain, lowest complexity for our shape, and the lowest managed-database cost — while being sufficient on scale. **AWS's real advantages** are the largest body of documentation and the most depth at extreme scale; neither is decisive now, and the documentation advantage applies to infrastructure configuration rather than to application code, which is where most of our AI-generated work happens. **Azure** has no decisive advantage for this platform, and would become the right answer only if a commercial agreement or a customer requirement made it so. Worth stating plainly: we already run on Google Cloud, so it is also the option with no migration cost — a legitimate practical factor, but recorded as that rather than dressed up as a technical finding.

One item in this report remains unverified and should be treated as reasoned judgement: the per-tenant schema approach combined with many running copies of the platform means a large number of open database connections, which is the most likely reason the database decision would need revisiting. The mobile and cloud findings in items four and five were checked against current published sources.